CARGO NEXUS

Autonomous Container Import Documentation & Port Release Engine

TechCrush Capstone | Group 2 | Project Documentation

1. Executive summary

Cargo Nexus is a proof-of-concept automation built with n8n. It ingests shipping documents, uses Gemini to extract structured data, stores records in PostgreSQL, compares document fields, routes unresolved discrepancies for human review, sends Gmail notifications, and produces an AI-assisted shipment readiness report. A FreightAPIs seaport lookup demonstrates an external data integration.

2. Problem and objective

Manual container document handling can cause delays when information is missing or inconsistent. The objective is to support faster preliminary review while preserving human oversight. This prototype does not make legally binding customs or terminal-release decisions.

3. Document scope

The intended scope covers Bill of Lading, Container Manifest, Commercial Invoice, Packing List, Form M, Customs Declaration, and Delivery Order. Demonstration reporting showed only some documents available; missing documents were flagged as outstanding.

4. System architecture

Intake webhook → PDF text extraction → Gemini-based structured extraction → normalization → PostgreSQL persistence → retrieval and discrepancy comparison. Downstream paths include document verification and notification, human approval/rejection and status updates, and Gemini-generated readiness reporting. A connected HTTP Request node queries FreightAPIs for port directory information.

5. Discrepancy and human review

The comparison stage examines normalized fields such as container identity and document details. A match in compared fields should be treated as a consistency result, not as confirmation that the full document set is complete. The exception path sends a human review request; approval and rejection paths update status and send notifications.

6. External integration

FreightAPIs seaport directory endpoint: GET /api/ports/CNSHA. Authentication is via Bearer token stored in an n8n credential. A standalone test returned JSON success=true for Shanghai, China. The endpoint currently uses a fixed demonstration port code rather than a shipment-derived code.

7. Tests and observations

During development, the port API returned structured JSON. Gmail verification/approval/rejection messages and an AI shipment readiness email were observed in individual tests. A sample readiness report flagged missing Packing List, Form M, Customs Declaration, and Delivery Order and stated NOT READY FOR CLEARANCE. The main workflow was published. A full post-integration end-to-end run across all branches has not been confirmed.

8. Security and reliability

API tokens and account credentials should remain in n8n credential storage and must not be included in exported files. Human approval gates support oversight. Error continuation is configured on the external HTTP node; the team should verify error handling and retry behavior before production deployment.

9. Limitations and next steps

The current workflow is a prototype. Add shipment-specific port lookup, verify completeness separately from field matching, test all branches end-to-end, validate database schema and duplicate handling, and add real customs/terminal integrations only with appropriate authorization.

10. Deliverables checklist

Included: README, project documentation, and active member list. Team must still add the actual n8n workflow JSON export and screenshots or other project assets. Review access permissions on the shared Google Drive folder before submitting.